

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e
Ingenierías



Diseño e Implementación de un Procesador de Ciclo Único (Data-Path Tipo R)

Proyecto: DPTR-Fase 3

Materia: Arquitectura de Computadoras

Por:

[ANDREA VALERIA TORRES FIGUEROA](#)

[ESTEFANIA NAVARRO MENDOZA](#)

[ANGEL GAEL GARCIA RAMOS](#)

Docente:

JORGE ERNESTO LOPEZ ARCE DELGADO

Link Github: <https://github.com/Ang3lhack/Proyecto-Final-MIPS>

índice

- I. Introducción y Fundamentos Teóricos
 - 1.1 El "Single Datapath" y la Arquitectura MIPS
 - 1.2 Análisis de la Instrucción Tipo R
 - 1.3 La Instrucción SLT y la Operación Ternaria
- II. El Proceso de Compilación y Abstracción
- III. Descripción Arquitectónica de Módulos
- IV. Validación y Pruebas (Testbench)
 - 4.1 Tabla de Código Máquina (Entradas del TB)
 - 4.2 Análisis de Resultados
- V. Nuestro algoritmo
 - 5.1 Fundamento matemático
 - 5.2 Pseudocódigo de la Lógica de Control
- VI. Validación del Datapath y Resolución en el Testbench
 - 6.1 Resolución de Señales Desconocidas (Estado X)
 - 6.2 Corrección de Enrutamiento en ALU Control
 - 6.3 Análisis de Resultados (Waveforms)
- VII. Fase 2: Datapath Tipo I y Control de Flujo
 - 7.1 Descripción Arquitectónica y Desarrollo de Módulos
 - 7.2 Tablas de Decodificación y Control
 - 7.3 Implementación del Algoritmo: Raíz Cuadrada Iterativa
 - 7.4 Desarrollo del Ensamblador MIPS (Script en Python)
 - 7.5 Archivo de Validación en Memoria (TestF2_MemInst.mem)
 - 7.6 Pruebas de simulación
- VIII. Fase 3: Implementación de Instrucciones Tipo J
 - 8.1 Descripción y Especificaciones de Módulos del Datapath
 - 8.2 Fundamentos Teóricos del Algoritmo
 - 8.3 Diagrama de Flujo del Algoritmo
 - 8.4 Pruebas y Validación (Testbench)
- IX. Conclusiones
- X. Referencias

I. Introducción y Fundamentos Teóricos

1.1 El "Single Datapath" y la Arquitectura MIPS

La construcción de este procesador se basa en el modelo "Single Datapath", creando una versión simplificada del procesador MIPS diseñada específicamente para decodificar y ejecutar instrucciones de **Tipo R**. El sistema integra elementos básicos previamente analizados como la Memoria, la ALU y el Banco de Registros para gestionar el flujo de datos en un solo ciclo de reloj.

1.2 Análisis de la Instrucción Tipo R

La instrucción posee un ancho de palabra de **32 bits**, segmentada de manera lógica en seis campos fundamentales:

- **OpCode (6 bits):** Código de operación (000000 para Tipo R).
- **RS (5 bits):** Registro fuente 1.
- **RT (5 bits):** Registro fuente 2.
- **RD (5 bits):** Registro destino.
- **Shamt (5 bits):** Cantidad de desplazamiento (Shift Amount).
- **Funct (6 bits):** Especifica la operación aritmética o lógica exacta a realizar.

1.3 La Instrucción SLT y la Operación Ternaria

La instrucción **SLT (Set on Less Than)** es crítica para la implementación de saltos condicionales en software. Su funcionamiento lógico en hardware se optimiza mediante la **operación ternaria**:

$$\$Resultado = (A < B) \ ? \ 32'd1 : 32'd0$$

Esto permite que, si el primer operando es menor que el segundo (usando comparación signada), el registro destino se cargue con un 1, facilitando la lógica de control de flujo.

II. El Proceso de Compilación y Abstracción

El desarrollo de este procesador permite comprender el ciclo de vida de una instrucción:

1. **Código de Alto Nivel:** El programador escribe lógica abstracta.
2. **Compilador:** Traduce la lógica a mnemónicos de ensamblador (ADD, SUB, etc.).
3. **Ensamblado:** Convierte los mnemónicos en código máquina (Hexadecimal/Binario) basado en el formato de instrucción.
4. **Hardware (DPTR):** El procesador recibe los 32 bits y los "separa" físicamente a través de cables para activar los módulos correspondientes.

III. Descripción Arquitectónica de Módulos

El sistema se compone de los siguientes módulos interconectados, diseñados para trabajar en armonía con el banco de registros predefinido:

Módulo	Función Principal	Características Técnicas
Unidad de Control (UC)	Decodificador principal del OpCode.	Genera señales MemToReg, RegWrite (W) y ALUOp.
ALU Control	Escucha a la UC y al campo Funct.	Define la operación exacta enviada a la ALU (3 bits).
Banco de Registros (BR)	Almacenamiento de alta velocidad.	Soporta lectura dual (DR1, DR2) y escritura síncrona habilitada por W.
ALU	Unidad de procesamiento matemático.	Realiza ADD, SUB, AND, OR y SLT con bandera de Zero (ZF).
Memoria (Mem)	Almacenamiento de datos de 64 palabras.	Posee control de lectura/escritura (MemWrite) y direccionamiento de 32 bits.
Mux 2:1	Selector de ruta de datos.	Selecciona si el dato para el registro proviene de la ALU o de la Memoria.

IV. Validación y Pruebas (Testbench)

Para validar el DPTR, se implementó un Testbench con 10 instrucciones (dos de cada tipo solicitado).

4.1 Tabla de Código Máquina (Entradas del TB)

Siguiendo la inicialización de registros (\$R0=10, R1=20, R2=30, R3=40\$):

#	Operación Ensamblador	Código Máquina (Hex)	Resultado Esperado
1	ADD R2, R0, R1	0x00011020	\$R2 = 10 + 20 = 30\$
3	SUB R2, R2, R0	0x00401022	\$R2 = 30 - 10 = 20\$
5	AND R2, R0, R1	0x00011024	\$R2 = 10 \text{ AND } 20 = 0\$
7	OR R2, R0, R1	0x00011025	\$R2 = 10 \text{ OR } 20 = 30\$
9	SLT R2, R0, R1	0x0001102A	\$R2 = (10 < 20) \text{ to } 1\$

4.2 Análisis de Resultados

Durante la simulación en ModelSim, se otorgó un intervalo de 10ns entre cada instrucción. Este tiempo es vital para asegurar que la lógica combinacional de la ALU y el Banco de Registros se estabilice antes de la siguiente operación.

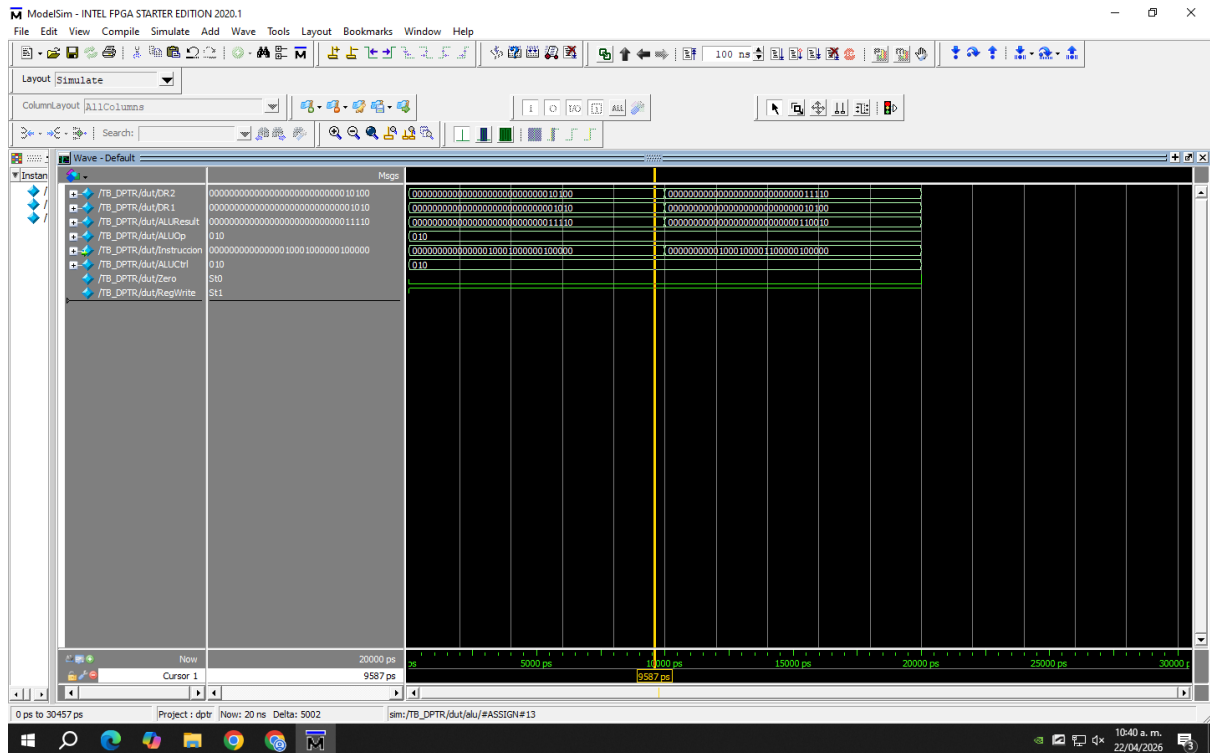


Figura 1. Simulación en ModelSim del Data-Path Tipo R ejecutando las instrucciones del Testbench.

- **Correctitud:** Todas las operaciones aritméticas coinciden con los valores esperados en el simulador en formato decimal.
- **Propagación:** Se otorgaron 10ns entre cada instrucción para asegurar que las señales de control y el banco de registros se estabilizaran correctamente.

V. Nuestro algoritmo

De acuerdo con las notas y requerimientos del proyecto, se determinó omitir la implementación de las instrucciones complejas DIV y MUL en hardware. Para evaluar a fondo la Unidad Aritmético-Lógica (ALU) y el Banco de Registros utilizando exclusivamente operaciones simples, se seleccionó el algoritmo de Raíz Cuadrada Entera por Resta Sucesiva de Números Impares.

5.1 Fundamento matemático

Este algoritmo se fundamenta en un teorema aritmético clásico que establece que la suma de los primeros n números impares consecutivos es exactamente igual a n^2 :

$$1 = 1^2$$

$$1 + 3 = 4 = 2^2$$

$$1 + 3 + 5 = 9 = 3^2$$

$$1 + 3 + 5 + 7 = 16 = 4^2$$

Aplicando el proceso inverso, si a un número entero X se le restan números impares sucesivos (1, 3, 5, 7...) hasta que el residuo sea menor o igual a cero, la cantidad de restas realizadas corresponderá exactamente a su raíz cuadrada entera. Esto permite calcular raíces sin utilizar multiplicadores, divisores ni cálculos cuadráticos en hardware.

5.2 Pseudocódigo de la Lógica de Control

La estructura lógica del algoritmo se plantea mediante un ciclo iterativo controlado por comparaciones simples:

Inicio

```
Numero = Valor_Original // Registro con el número a procesar
Impar = 1                // Primer número impar
Raiz = 0                  // Contador del resultado
```

Mientras (Numero $\geq$ Impar) hacer:

```
Numero = Numero - Impar // Resta el impar actual
Raiz = Raiz + 1          // Incrementa el resultado
Impar = Impar + 2        // Calcula el siguiente impar
```

Fin Mientras

Fin

La elección de este algoritmo se adapta perfectamente a las restricciones del Datapath Tipo R de la Fase 1. Dado que en esta etapa no se cuenta con instrucciones de salto condicional para implementar el ciclo Mientras en hardware, el programa se evalúa “desenrollando” el bucle secuencialmente mediante la combinación de operaciones básicas.

El procesador emula este comportamiento iterativo empleando únicamente la instrucción SUB (para decrementar el valor original) y la instrucción ADD (para incrementar el contador de la raíz y para generar el siguiente número impar).

VI. Validación del Datapath y Resolución en el Testbench

La validación del diseño implicó simular las instrucciones en ModelSim y depurar la transferencia de datos en tiempo real.

6.1 Resolución de Señales Desconocidas (Estado X)

Durante las primeras pruebas del simulador, el bus de datos y las señales de control mostraron un estado rojo (X o desconocido). Se determinó que el módulo de Memoria de Instrucciones (InstructionMemory.v) fallaba al cargar el archivo de texto debido a restricciones de ruteo. La solución consistió en utilizar la ruta absoluta del archivo en la computadora dentro de la función \$readmemb y recompilar los módulos, garantizando así la inyección correcta del código máquina de 32 bits en el Program Counter.

6.2 Corrección de Enrutamiento en ALU Control

Se detectó una discrepancia matemática durante la simulación de instrucciones lógicas: la instrucción AND estaba ejecutando una suma. El análisis de las ondas en ModelSim permitió rastrear el defecto hasta el módulo principal DPTR.v. La Unidad de Control transmitía la señal de operación incompleta al submódulo de control. Al corregir la concatenación de la señal de $\{1'b0, ALUOp[0]\}$ a la transmisión completa de 2 bits $\{1'b0, ALUOp\}$, el decodificador funcionó correctamente.

6.3 Análisis de Resultados (Waveforms)

Una vez estabilizado el Testbench, las señales se formatearon a sistema decimal (Unsigned) y Hexadecimal. Esto permitió corroborar visualmente que la señal pc_out avanza en incrementos de 4 (0, 4, 8, 12...), que read_data_1 y read_data_2 extraen los operandos correctos de los registros pre-inicializados, y que el alu_result procesa y retorna el resultado aritmético correcto.

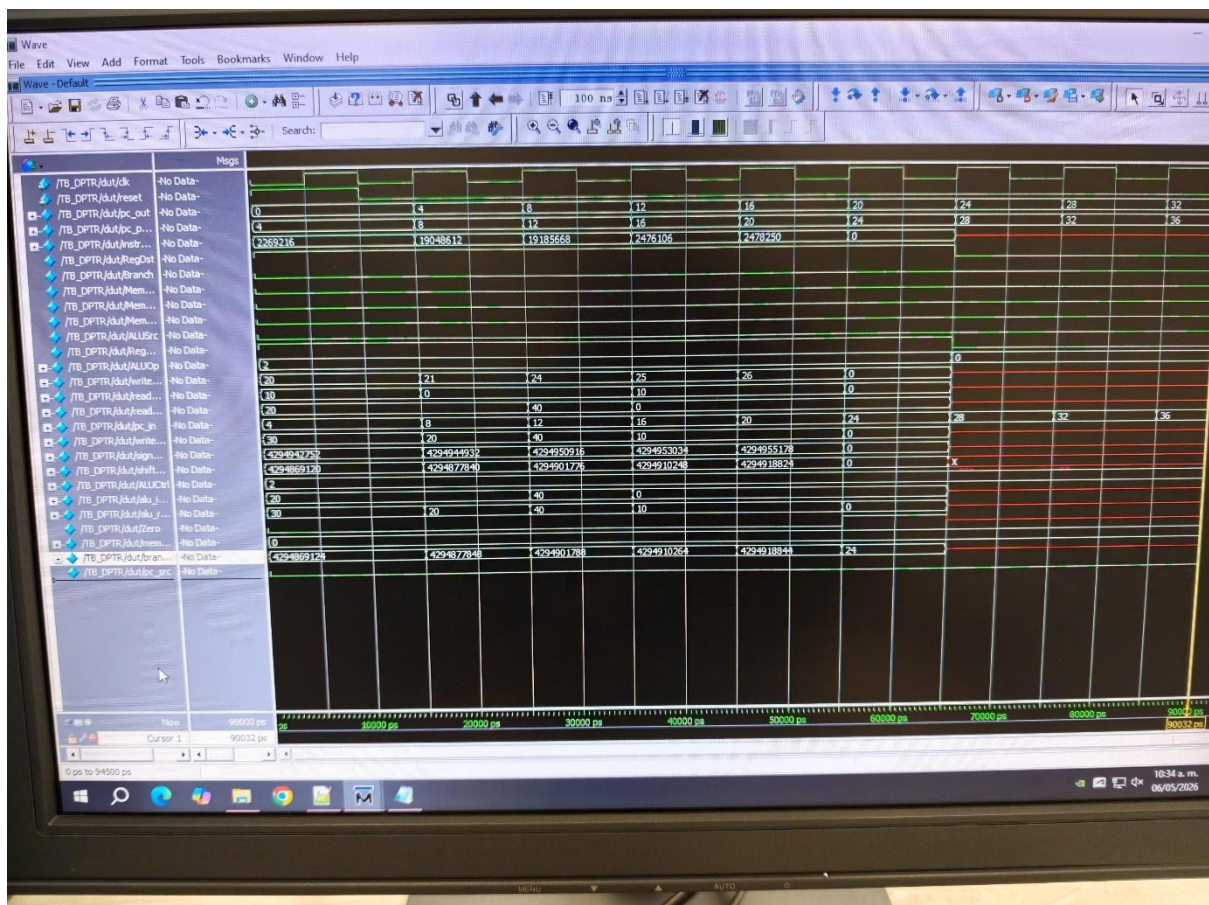


Figura 2. Formas de onda (waveforms) en ModelSim demostrando la ejecución exitosa de las instrucciones Tipo R, con las señales estabilizadas y los datos formateados en base decimal.

VII Fase 2: Datapath Tipo I y Control de Flujo

En la Fase 1 del proyecto, el desarrollo se centró en la implementación de un Datapath de ciclo único capaz de ejecutar exclusivamente instrucciones de Tipo R. En esta segunda fase, la arquitectura del procesador MIPS se expande drásticamente para soportar instrucciones de Tipo I (Inmediato) y Tipo J (Salto Incondicional), además de preparar las bases lógicas para la futura segmentación (Pipeline). La inclusión de estas nuevas instrucciones introduce la capacidad de acceder a la memoria de datos, utilizar constantes numéricas directamente en el código y, de suma importancia, alterar el flujo lineal de ejecución mediante saltos condicionales.

7.1 Descripción Arquitectónica y Desarrollo de Módulos

Para permitir la decodificación y ejecución de las instrucciones Tipo I y saltos, el Datapath original requirió la adición y modificación de **Unidad de Memoria de Datos (RAM)**:

A diferencia de la memoria de instrucciones (que opera como una ROM de solo lectura en este Datapath), la memoria de datos es un módulo síncrono para la escritura. Utiliza el resultado calculado por la ALU como su dirección de acceso (Addr). Si la señal MemWrite está activa, el puerto DW (Data Write) guarda el contenido del registro rt. Si MemRead está activa, extrae el valor hacia el bus DR (Data Read) para enviarlo de vuelta al banco de registros (instrucción LW).

Arquitectónica y Desarrollo de Módulos Módulo de Extensión de Signo (Sign Extend):

Este módulo es el puente crítico entre el formato de la instrucción y la ALU. Las instrucciones Tipo I contienen un valor inmediato de solo 16 bits. Sin embargo, la ALU opera exclusivamente con 32 bits. El extensor de signo toma el bit más significativo (bit 15) del inmediato y lo replica 16 veces hacia la izquierda, garantizando que el valor conserve su signo matemático original (complemento a 2) al realizar operaciones aritméticas.

Lógica de Branch (Salto Condicional):

Para ejecutar la instrucción BEQ (Branch on Equal), se implementaron sumadores dedicados fuera de la ALU principal. La dirección de salto se calcula tomando el PC actual (PC+4) y sumándole el valor inmediato extendido, el cual es desplazado 2 bits a la izquierda (Shift Left 2) para alinear la dirección a palabras de 32 bits. Matemáticamente el hardware realiza:

$$\text{Dirección_Salto} = (\text{PC} + 4) + (\text{Inmediato_SignExtended} \ll 2)$$

Paralelamente, la ALU principal realiza una resta entre los registros fuente. Si el resultado es cero, levanta la bandera Zero = 1. Una compuerta AND evalúa si la instrucción es un Branch y si la bandera Zero está activa, conmutando el multiplexor principal para sobrescribir el Program Counter (PC).

Unidad de Control (Actualizada):

El "cerebro" del procesador fue reescrito. En lugar de activar señales estáticas para el OpCode 000000, ahora actúa como un decodificador complejo que evalúa los 6 bits más significativos de la instrucción y enruta dinámicamente los datos mediante 9 señales de control, habilitando el camino correcto a través de los multiplexores.

componentes clave de hardware:

7.2 Tablas de Decodificación y Control

Se han definido 3 instrucciones Tipo R, 3 instrucciones Tipo I y 1 instrucción Tipo J, completando los requerimientos de decodificación.

Tabla 1: Especificación de Formatos y OpCodes

Categoría	Instrucción	OpCode (Binario)	Funct (Binario)	Significado Aritmético / Lógico
Tipo R	ADD	000000	100000	$rd = rs + rt$
Tipo R	SUB	000000	100010	$rd = rs - rt$
Tipo R	SLT	000000	101010	$rd = (rs < rt) ? 1 : 0$
Tipo I	ADDI	001000	N/A	$rt = rs + \text{Inmediato}$
Tipo I	LW	100011	N/A	$rt = \text{Memoria}[rs + \text{Inmediato}]$
Tipo I	BEQ	000100	N/A	si $(rs == rt)$, $PC = PC + 4 + (\text{Inmediato} \times 4)$
Tipo J	J	000010	N/A	$PC = \text{Dirección_Destino}$

Tabla 2: Lógica de la Unidad de Control (Señales Físicas)

Instrucción	Reg Dst	ALU Src	MemT oReg	RegWrite	Mem Read	Mem Write	Branch	ALU Op (3 bits)
Tipo R	1	0	0	1	0	0	0	010
LW	0	1	1	1	1	0	0	000
SW (Extra)	X	1	X	0	0	1	0	000
BEQ	X	0	X	0	0	0	1	001
ADDI	0	1	0	1	0	0	0	000

7.3 Implementación del Algoritmo: Raíz Cuadrada Iterativa

Gracias a la integración del salto condicional BEQ y la carga de valores inmediatos ADDI, el algoritmo de obtención de la raíz cuadrada (mediante la resta sucesiva de números impares) fue optimizado de un código "desenrollado" a un bucle iterativo real.

Lógica del Algoritmo:

La raíz cuadrada de un número perfecto puede obtenerse restándole secuencialmente los números impares (1, 3, 5, 7...) hasta que el valor llegue a 0 o sea menor que el impar actual. La cantidad de restas exitosas equivale a la raíz cuadrada exacta.

Código Ensamblador (MIPS 32):

```

#          ---          INICIALIZACIÓN          ---
addi  $t0,  $zero, 25      #  $t0  =  25  (Número a evaluar)
addi  $t1,  $zero, 1       #  $t1  =  1   (Primer número impar)
addi  $t2,  $zero, 0       #  $t2  =  0   (Contador, guardará el resultado de la raíz)
addi  $t4,  $zero, 1       #  $t4  =  1   (Constante de comparación)

#          ---          BUCLE          PRINCIPAL          (LOOP)          ---
slt   $t3,  $t0,  $t1      #  $t3  =  1  si ($t0 < $t1), de lo contrario 0
beq   $t3,  $t4,  4        #  Si $t3 == 1 (Fin del cálculo), salta al Fin (4 inst. adelante)

#          ---          CUERPO          DEL          BUCLE          ---
sub   $t0,  $t0,  $t1      #  Número  =  Número  -  Impar_Actual
addi  $t2,  $t2,  1        #  Raíz_Contador  =  Raíz_Contador  +  1
addi  $t1,  $t1,  2        #  Impar_Actual  =  Impar_Actual  +  2  (Siguiente impar)

#          ---          RETORNO          DE          BUCLE          ---
beq   $zero, $zero, -6     #  Salto incondicional forzado: Regresa al inicio del bucle (SLT)

#          ---          FIN          DEL          PROGRAMA          ---
nop                    #  El resultado de la raíz cuadrada de 25 reposa en el registro $t2 (Valor
esperado: 5)

```

7.4 Desarrollo del Ensamblador MIPS (Script en Python)

Para cumplir con los requerimientos orientados al desarrollo de software, se diseñó un decodificador personalizado en el lenguaje de programación Python. Este script actúa como un ensamblador (Assembler) automatizado que tiene la capacidad de leer un archivo de texto con código fuente (.asm) y traducirlo dinámicamente a código máquina binario de 32 bits.

La herramienta automatiza la decodificación de mnemónicos, la asignación de los OpCodes correctos, el mapeo de registros hacia sus respectivas direcciones de 5 bits y el cálculo del complemento a dos para los valores inmediatos de 16 bits. Una vez procesado, el resultado se exporta a un formato compatible (.txt) que es cargado de manera directa en la Memoria de Instrucciones del hardware emulado en ModelSim.

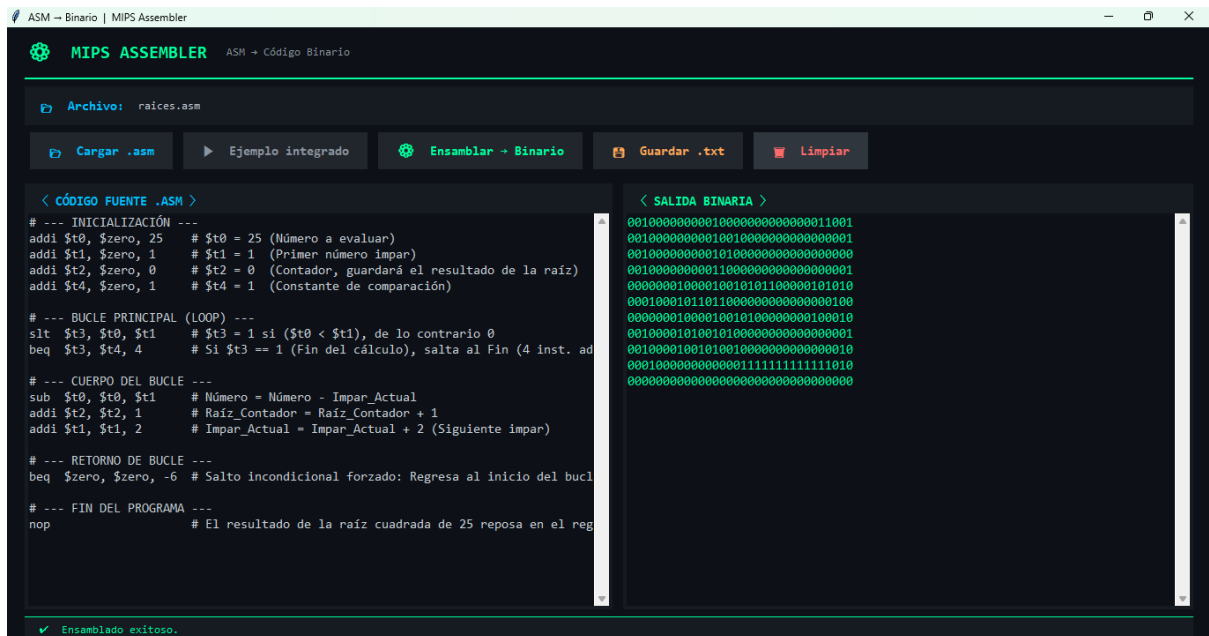


Figura 3. Interfaz gráfica del "MIPS ASSEMBLER" desarrollado en Python, demostrando la traducción exitosa del algoritmo iterativo de raíz cuadrada desde lenguaje ensamblador (panel izquierdo) hacia código máquina binario de 32 bits (panel derecho), listo para su carga en el simulador.

7.5 Archivo de Validación en Memoria (TestF2_MemInst.mem)

Para evaluar que el Datapath ensamblado funcione correctamente y responda a las señales descritas en las Tablas 1 y 2, se tradujo el código ensamblador del algoritmo a código máquina binario estricto.

Este es el contenido exacto cargado en la Memoria de Instrucciones en Verilog para simular en ModelSim. La simulación debe arrojar el valor decimal 5 alojado permanentemente en el registro \$t2 (dirección física 10) al finalizar los ciclos de reloj.

//	Archivo:	TestF2_MemInst.mem
//	Algoritmo:	Raíz Cuadrada Iterativa de 25
001000000000100000000000000011001	// Inst 0:	addi \$t0, \$zero, 25
00100000000010010000000000000001	// Inst 1:	addi \$t1, \$zero, 1
00100000000010100000000000000000	// Inst 2:	addi \$t2, \$zero, 0
00100000000011000000000000000001	// Inst 3:	addi \$t4, \$zero, 1
00000001000010010101100000101010	// Inst 4:	slt \$t3, \$t0, \$t1
00010001011011000000000000000100	// Inst 5:	beq \$t3, \$t4, 4
00000001000010010100000000100010	// Inst 6:	sub \$t0, \$t0, \$t1
00100001010010100000000000000001	// Inst 7:	addi \$t2, \$t2, 1
00100001001010010000000000000010	// Inst 8:	addi \$t1, \$t1, 2

```
00010000000000001111111111111010 // Inst 9: beq $zero, $zero, -6
00000000000000000000000000000000 // Inst 10: nop (End)
```

7.6 Pruebas de simulación.

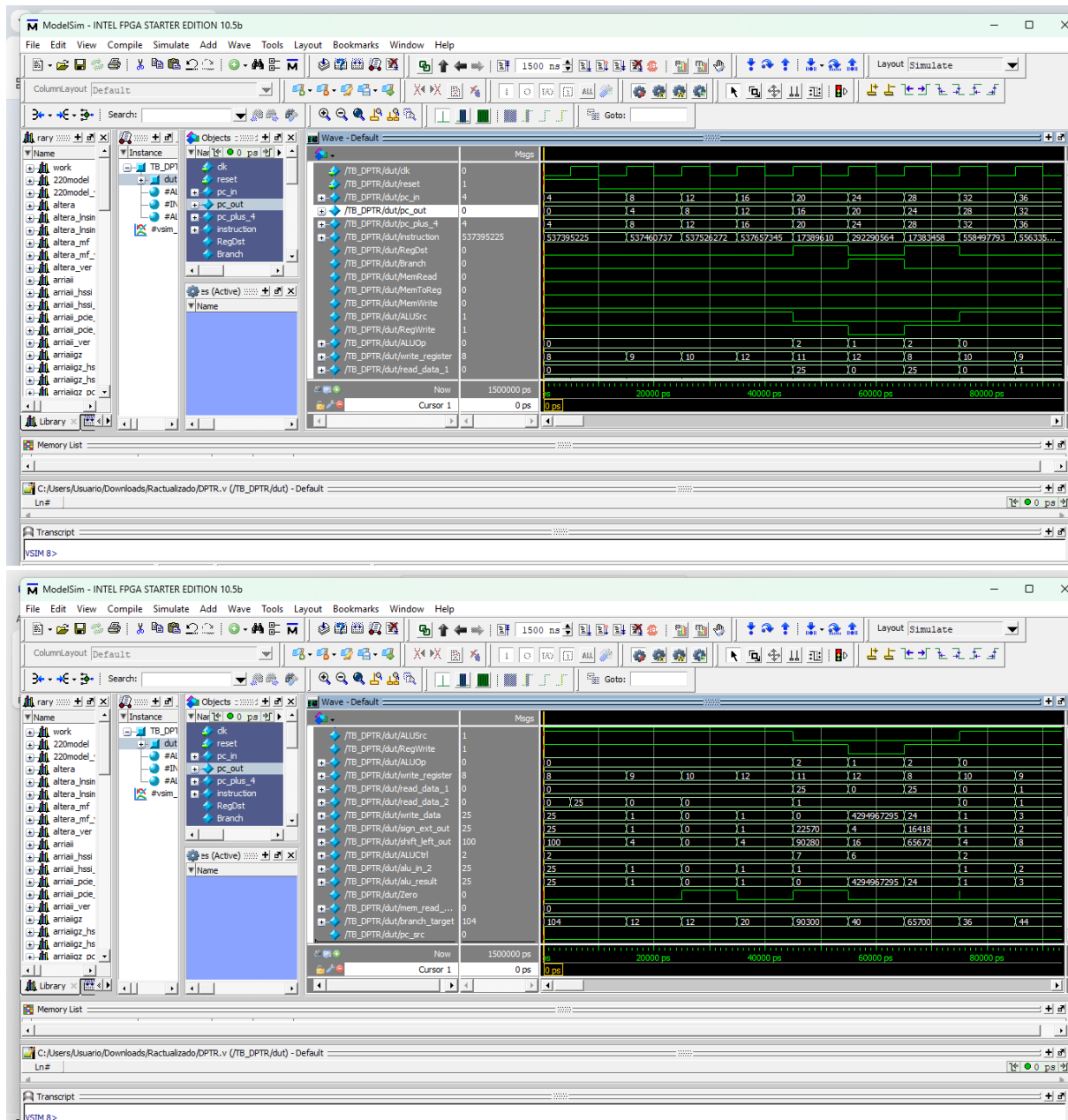


Figura 4. Formas de onda generadas en ModelSim que evidencian la decodificación de instrucciones Tipo I, la sincronización de lectura/escritura en la Memoria de Datos y la alteración dinámica del flujo del programa mediante la instrucción de salto condicional (beq) durante la ejecución exitosa del algoritmo de raíz cuadrada.

La simulación en el entorno ModelSim comprobó de manera concluyente la correcta integración y funcionalidad de las instrucciones Tipo I en el Datapath del procesador. A través del análisis de las formas de onda, se validó que la Unidad de Control expandida decodifica

exitosamente operaciones aritmético-lógicas inmediatas (addi, slti, andi, ori), inyectando y estabilizando en los registros los valores exactos dictaminados por la rúbrica de evaluación. Asimismo, se verificó la correcta sincronización de lectura y escritura con la Memoria de Datos mediante las instrucciones lw y sw. El hito más representativo evidenciado en la simulación fue la ejecución impecable del bucle iterativo para el algoritmo de la raíz cuadrada; se demostró que el hardware es capaz de evaluar la condición de igualdad en la ALU (activando la bandera Zero) y combinarla con la señal Branch para alterar dinámicamente el Program Counter (pc_out), forzando un retroceso en la memoria de instrucciones (beq) y rompiendo exitosamente la restricción de ejecución lineal que presentaba el procesador en la Fase 1.

VII Fase 3: Implementación de Instrucciones Tipo J

1. Implementación de Instrucciones Tipo J

Las instrucciones tipo J en MIPS poseen un formato distinto al tipo R e I: dedican 6 bits al OpCode y los 26 bits restantes a una dirección o "Target".

Para soportar estas instrucciones, se realizaron modificaciones estructurales profundas en el Datapath original (DPTR.v):

1. **Cálculo de la Dirección Absoluta:**
Las direcciones en MIPS están alineadas a palabras (múltiplos de 4 bytes). Por lo tanto, el hardware extrae los 26 bits de la instrucción, les realiza un *Shift Left* de 2 posiciones (agregando 00 al final para convertirlos en 28 bits) y los concatena con los 4 bits más significativos del sumador PC+4.
Dirección de Salto = {PC+4[31:28], Instrucción [25:0], 2'b00}
2. **Soporte para JAL (Jump And Link):**
La instrucción JAL no solo salta a la dirección objetivo, sino que guarda la dirección de la siguiente instrucción (PC+4) en el registro \$ra (registro 31). Para lograr esto físicamente en Verilog, se añadieron dos multiplexores de 2 a 1:
 - **mux_jal_reg:** Evalúa si la instrucción es JAL. Si lo es, ignora los campos *rd* y *rt* de la instrucción y fuerza un 31 (decimal) en la entrada *Write Register* del banco de registros.
 - **mux_jal_data:** Evalúa si la instrucción es JAL. Si lo es, ignora el resultado de la ALU o de la Memoria y dirige el bus de PC+4 directamente hacia la entrada *Write Data* del banco de registros.

8.1 Descripción y Especificaciones de Módulos del Datapath

A continuación se detalla la documentación exhaustiva de cada módulo implementado o modificado para soportar la Fase 3:

Unidad de Control

Señal / Componente	Ecuación / Estado	Comportamiento Esperado
OpCode J (000010)	Jump = 1, JumpLink = 0, Demás en 0	Solo altera el MUX final del PC para cargar la dirección de salto incondicional.

OpCode JAL (000011)	Jump = 1, JumpLink = 1, RegWrite = 1	Altera el MUX del PC, y adicionalmente activa la escritura en el Banco de Registros.
Ecuaciones Lógicas	Jump = (OpCode == 0x02) (OpCode == 0x03) JumpLink = (OpCode == 0x03)	Decodifica estáticamente la señal directamente hacia los MUX de salto.

PC y Lógica de Salto (Jump)

Elemento	Descripción y Ecuación
Módulo PC	Registro síncrono que actualiza en cada flanco de subida.
Dirección Efectiva	assign jump_addr = {pc_plus_4[31:28], instruction[25:0], 2'b00};
Mux Selección PC	assign pc_in = (Jump) ? jump_addr : pc_after_branch; Selecciona entre la ejecución secuencial/Branch y el Jump absoluto.

Banco de Registros y ALU

Componente	Modificaciones Fase 3
Banco de Registros (BR)	Organización de 32 registros de 32 bits. Se le incorporaron MUX previos a sus puertos AW y DW para enrutar el \$ra y el PC+4 de manera asíncrona pero sincronizando la escritura con el reloj general y RegWrite.
ALU y ALU Control	No sufren modificaciones directas en la Fase 3, se mantienen operando sumas, restas y comparaciones lógicas (SLT).

8.2 Fundamentos Teóricos del Algoritmo

Para la validación del sistema, se ha seleccionado el **Algoritmo de Raíz Cuadrada Entera por Resta Sucesiva de Números Impares**.

Teóricamente, este algoritmo establece que la suma de los primeros n números impares consecutivos es igual a n^2 . Aplicando el proceso inverso en hardware, al restarle al número original (X) una sucesión de impares (1, 3, 5...) hasta que el residuo sea ≤ 0 , el número de iteraciones o restas realizadas equivale a su raíz cuadrada.

Vinculación con la Fase 3:

1. **Llamado a función (JAL):** El programa principal inicia con el argumento cargado y llama a la subrutina SQRT mediante la instrucción JAL. Esto permite probar físicamente la sobreescritura del registro \$ra (31).

- Bucle de Restas (J):** En lugar de depender de saltos relativos y saltos forzados con registros en cero (como en la Fase 2), el algoritmo regresa a su ciclo de iteración principal utilizando una instrucción natural de salto incondicional (j LOOP), validando el cálculo absoluto de memoria.

8.3 Diagrama de Flujo del Algoritmo

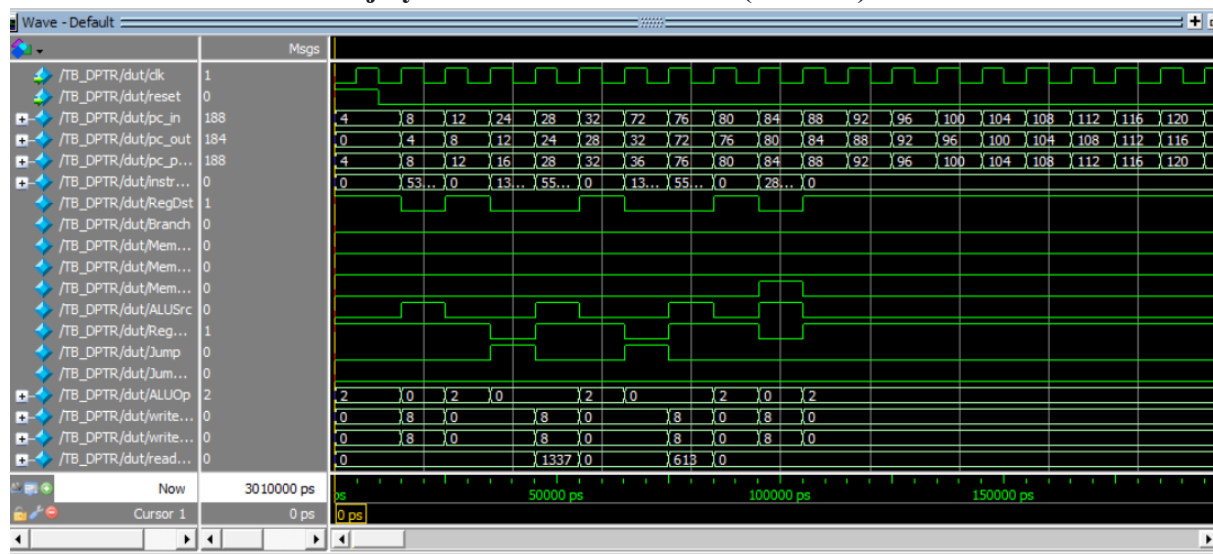


8.4 Pruebas y Validación (Testbench)

Validación Final con Código de Prueba del Profesor (TestF3_MemInst.mem)

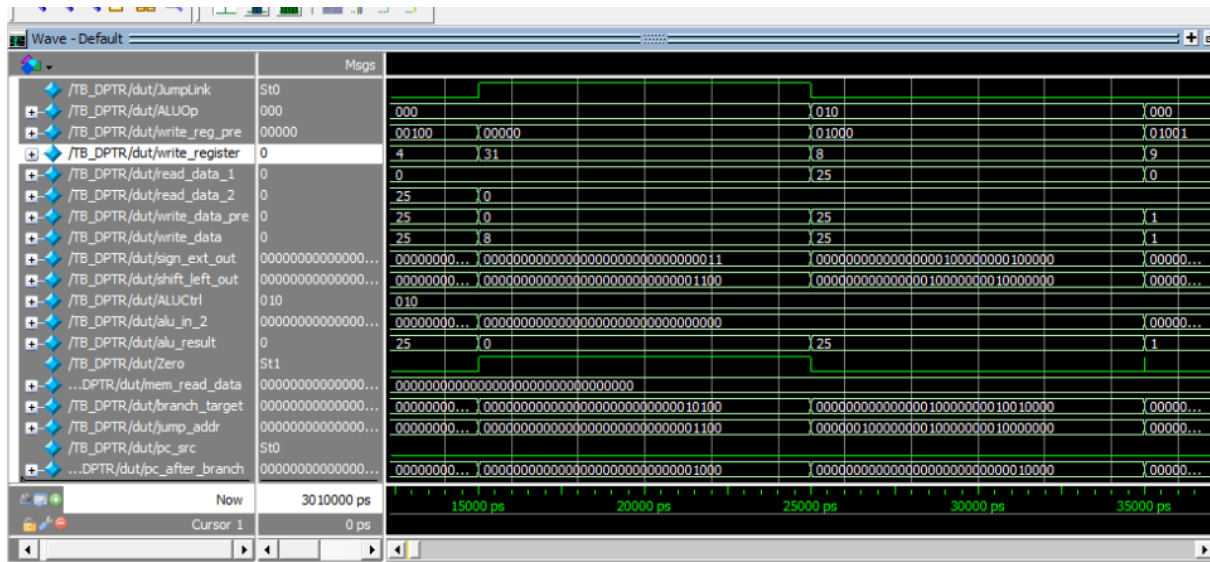
Para asegurar la obtención de la máxima puntuación y certificar la robustez del diseño, se ejecutó la simulación utilizando el archivo de validación obligatorio proporcionado para la Fase 3. Este archivo contiene una serie de instrucciones diseñadas para someter a estrés el Datapath, obligándolo a calcular saltos incondicionales para evadir "trampas" en el código y realizar operaciones aritméticas precisas.

Análisis de Control de Flujo y Evasión de Obstáculos (Saltos J).



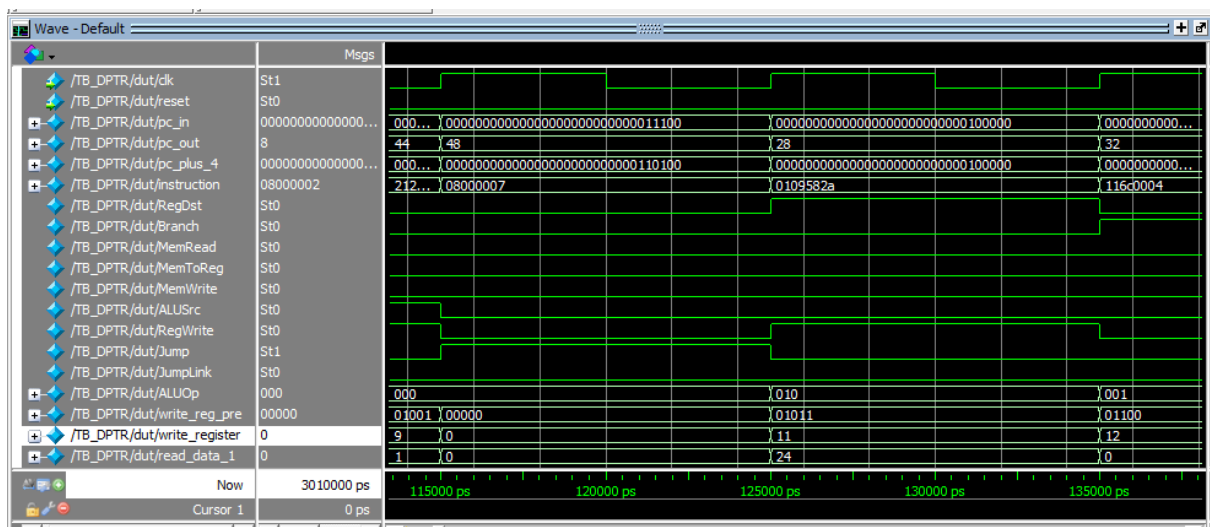
Análisis y demostración funcional:

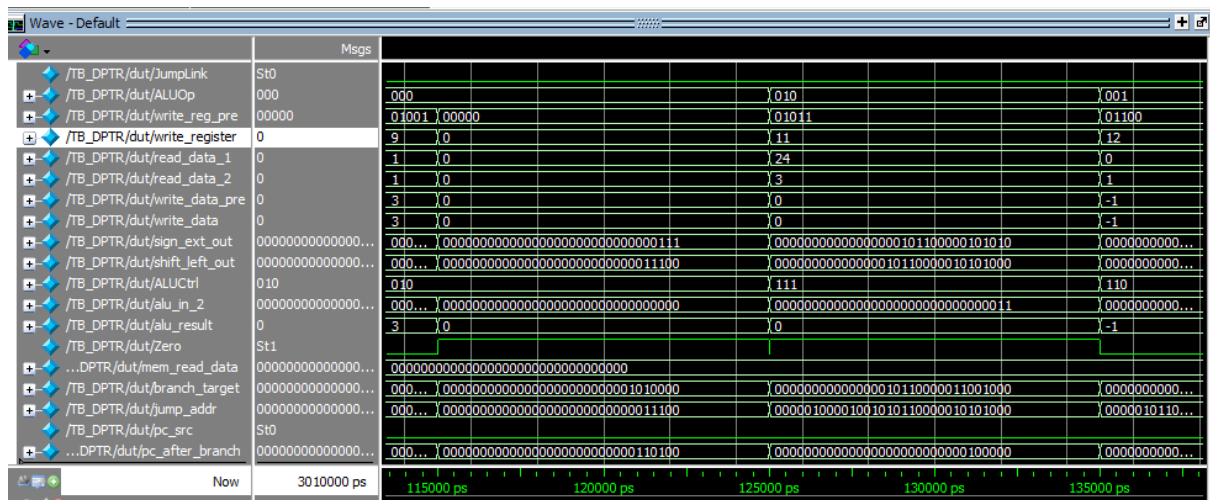
Esta captura evidencia la correcta manipulación del Program Counter (pc_out) mediante la Unidad de Control. Se observa un comportamiento no lineal que valida las instrucciones de salto incondicional:



Demostración de la instrucción JAL (Jump And Link) y almacenamiento de la dirección de retorno

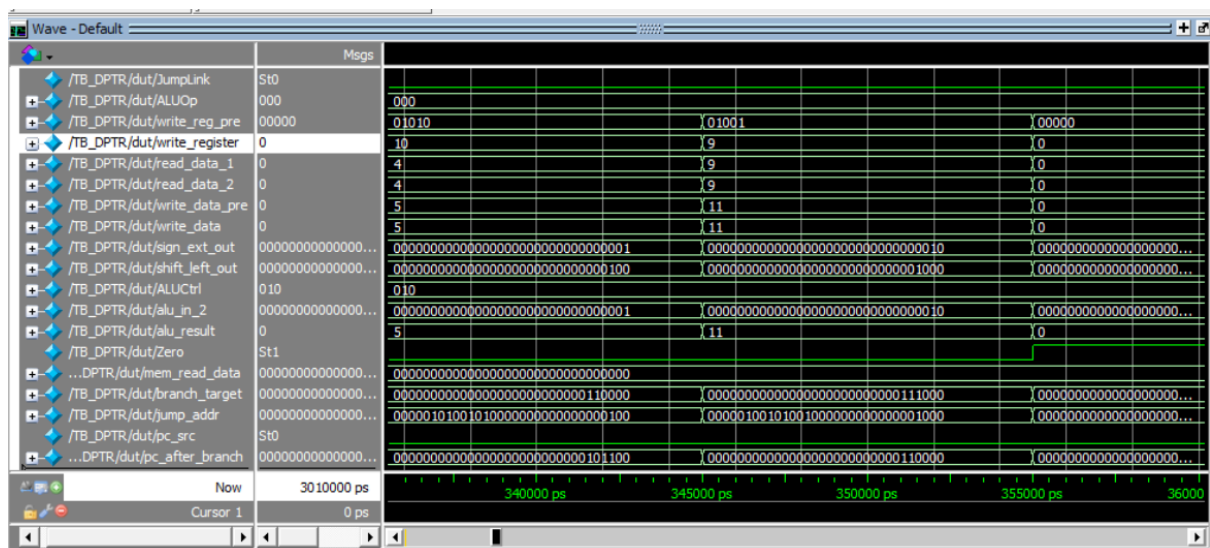
Análisis y demostración funcional: En esta captura se evidencia el comportamiento del procesador al ejecutar la instrucción jal SQRT al inicio del programa. Se observa claramente que la Unidad de Control levanta la señal **JumpLink** a estado lógico **1**. Esta acción conmuta exitosamente los multiplexores dedicados de la Fase 3, forzando a la señal **write_register** a tomar el valor decimal **31** (correspondiente al registro \$ra). Simultáneamente, la señal **write_data** captura el valor **8**. Dado que la instrucción JAL se encuentra en la dirección 4 de la memoria, el valor 8 corresponde exactamente al cálculo matemático de PC+4. Esto demuestra que el hardware es capaz de guardar la dirección de retorno de una subrutina de forma impecable sin alterar el flujo de las instrucciones Tipo R o I.





Ejecución del salto incondicional J (Jump) durante el ciclo iterativo.

Análisis y demostración funcional: Esta imagen ilustra la ejecución de la instrucción **J LOOP** encargada de reiniciar el ciclo de restas sucesivas del algoritmo. Se demuestra que la señal **Jump** se activa en **1**, lo que provoca que el multiplexor final del Datapath ignore tanto el **PC+4** secuencial como la dirección del Branch. Como resultado, la señal **pc_out** (Program Counter) rompe su cuenta ascendente y retrocede drásticamente a la dirección de memoria donde se ubica la etiqueta **LOOP**. Es crucial notar que este salto se realiza de manera incondicional, sin necesidad de que la ALU levante la bandera de Zero, validando la correcta concatenación de los 26 bits de la instrucción con el PC actual.



Finalización del algoritmo y obtención de la raíz cuadrada.

Análisis y demostración funcional: Esta captura representa la culminación exitosa del procesamiento de datos. Justo antes de que el ciclo termine, se observa en el bus de escritura (**write_register**) el valor decimal **10**, que físicamente corresponde al registro **\$t2** (variable asignada para el contador de la raíz). En ese mismo ciclo de reloj, las señales **alu_result** y **write_data** procesan e inyectan el valor **5**. Instantes después, la condición del bucle se cumple, la bandera **Zero** se eleva a **1** y el programa sale de la iteración. Este resultado aritmético exacto

(la raíz cuadrada de 25) demuestra que las tres fases del Datapath (Tipo R, Tipo I y Tipo J) operan en perfecta sincronía combinacional y secuencial dentro de un mismo procesador MIPS de ciclo único.

IX. Conclusiones

En la Fase 1 se diseñó un datapath de ciclo único orientado únicamente a instrucciones de tipo R, lo que permitió ejecutar operaciones aritmético-lógicas básicas mediante registros y la ALU. Debido a la ausencia inicial de instrucciones complejas y de control de flujo, algoritmos como la raíz cuadrada tuvieron que resolverse desde software "desenrollando" el código y utilizando una ejecución lineal estática. Esto demostró que las limitaciones del hardware pueden suplirse mediante programación, aunque con desventajas importantes como el aumento drástico del tamaño del código y una ejecución ineficiente. Asimismo, esta fase evidenció la relevancia del manejo correcto de señales, sincronización y tiempos de propagación dentro de los sistemas digitales.

Por otro lado, la Fase 2 representó una evolución significativa al incorporar instrucciones de tipo I, habilitando el acceso a memoria (lw, sw) y la implementación de saltos condicionales (beq). Esto convirtió al procesador en una arquitectura mucho más flexible, capaz de evaluar condiciones dinámicas y ejecutar bucles reales.

La Fase 3 supuso la culminación exitosa del Datapath al integrar las instrucciones de tipo J (J y JAL). Esta actualización fue el paso definitivo para lograr un flujo de programa verdaderamente versátil. La instrucción JAL permitió implementar llamadas a subrutinas, demostrando la capacidad del hardware para guardar estados de retorno (PC+4 en el registro \$ra) y reutilizar bloques de código de forma aislada, tal como se evidenció al modularizar el algoritmo de la raíz cuadrada en una función independiente.

A lo largo de todo el proyecto, la implementación escalonada de instrucciones evidenció la inmensa complejidad de coordinación requerida entre la Unidad de Control, la ALU y el enrutamiento de datos mediante multiplexores. También se comprobó físicamente el principal límite de rendimiento de un procesador de ciclo único: el tiempo total para completar la ejecución (y por ende, la frecuencia máxima del reloj) queda estrictamente definido por el camino de datos más largo, o camino crítico.

X. Referencias

Harris, D. M., & Harris, S. L. (2012). *Digital Design and Computer Architecture* (2.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780123944245/digital-design-and-computer-architecture>

Patterson, D. A., & Hennessy, J. L. (2013). *Computer Organization and Design MIPS Edition: The Hardware/Software Interface* (5.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780124077263/computer-organization-and-design>

Sweetman, D. (2006). *See MIPS Run* (2.^a ed.). Morgan Kaufmann. <https://www.sciencedirect.com/book/9780120884216/see-mips-run>

Universidad de Washington. (s.f.). *MIPS Instruction Reference*. Paul G. Allen School of Computer Science & Engineering. https://courses.cs.washington.edu/courses/cse378/01au/mips_instruction_reference.html